

**TRƯỜNG ĐẠI HỌC SƯ PHẠM TP.HCM
KHOA TOÁN – TIN HỌC**

TÓM TẮT BÀI GIẢNG

Môn

LÝ THUYẾT ĐỒ THỊ

Giảng viên biên soạn: Nguyễn Ngọc Trung

TP.HCM 9.2006

MỤC LỤC

Chương 1. Đại cương về đồ thị	3
1.1 Giới thiệu	3
1.2 Định nghĩa đồ thị.....	3
1.3 Một số thuật ngữ cơ bản	6
1.4 Đường đi, chu trình và đồ thị liên thông	8
1.5 Biểu diễn đồ thị trên máy tính	11
1.5.1 Biểu diễn đồ thị bằng ma trận kề	11
1.5.2 Ma trận liên thuộc đỉnh – cạnh	13
Chương 2. Đồ thị Euler và đồ thị Hamilton.....	15
2.1 Đồ thị Euler.....	15
2.2 Đồ thị Hamilton.....	17
Chương 3. Đồ thị có trọng số và bài toán đường đi ngắn nhất.....	20
3.1 Đồ thị có trọng số	20
3.2 Bài toán đường đi ngắn nhất.....	21
3.2.1 Đường đi ngắn nhất xuất phát từ một đỉnh.....	22
3.2.2 Thuật toán Dijkstra – Trường hợp đồ thị không có cạnh/cung âm.....	25
Chương 4. Cây.....	27
4.1 Định nghĩa và tính chất của cây	27
4.2 Cây khung và bài toán cây khung nhỏ nhất.....	29
4.2.1 Thuật toán Kruskal:.....	30
4.2.2 Thuật toán Prim.....	31
TÀI LIỆU THAM KHẢO	34

Chương 1.

Đại cương về đồ thị

1.1 Giới thiệu

Lý thuyết đồ thị là một lĩnh vực nghiên cứu đã có từ lâu và có nhiều ứng dụng trong ngành công nghệ thông tin. Những tư tưởng cơ bản của lý thuyết đồ thị được đề xuất vào những năm đầu của thế kỷ 18 bởi nhà toán học lỗi lạc người Thụy Sĩ: Leonhard Euler. Chính ông là người đã sử dụng đồ thị để giải bài toán nổi tiếng về 7 cái cầu ở thành phố Konigberg.

Những ứng dụng cơ bản của đồ thị như:

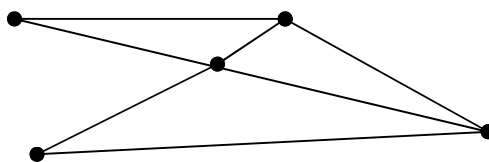
- Xác định tính liên thông trong một mạng máy tính: hai máy tính nào đó có thể truyền dữ liệu cho nhau được không.
- Tìm đường đi ngắn nhất trên mạng giao thông
- Giải các bài toán tối ưu: lập lịch, phân bổ tần số cho các trạm phát thanh, truyền hình.
- Giải bài toán tô màu trên bản đồ: tìm số màu ít nhất để tô các quốc gia sao cho hai quốc gia kề nhau phải được tô khác màu.
- ...

1.2 Định nghĩa đồ thị

Một cách trực quan, ta có thể hình dung đồ thị là một cấu trúc rời rạc gồm tập hợp các đỉnh và tập hợp các cạnh nối các đỉnh đó. Có nhiều loại đồ thị khác nhau biểu diễn cho những đối tượng khác nhau và trong các ứng dụng khác nhau.

Người ta phân loại đồ thị dựa trên đặc điểm của các cạnh nối. Cụ thể hơn ta xét một bài toán cụ thể trong đó có sử dụng đồ thị để mô hình hóa bài toán: “Mô hình hệ thống giao thông tại một thành phố và xây dựng các ứng dụng như tìm đường đi, tìm kiếm địa chỉ, ...”.

Để mô hình hệ thống giao thông trên, ta có thể biểu diễn mỗi địa điểm (giao lộ, trung tâm, ...) là một điểm và các con đường nối các giao lộ sẽ là các cạnh như trong hình dưới đây:



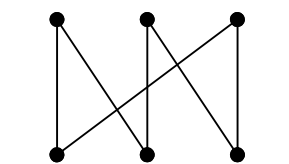
Trong cách biểu diễn này, ta thấy nhiều nhất chỉ có một con đường nối hai địa điểm trực tiếp với nhau, các con đường đều là hai chiều và không có con đường nào nối một địa điểm với chính nó. Và đồ thị biểu diễn mô hình này cũng phải thỏa mãn các tính chất trên. Dạng đồ thị như vậy là gọi là: **đơn đồ thị vô hướng**.

Định nghĩa 1.1. Một đơn đồ thị vô hướng là một bộ $G=\langle V,E \rangle$, trong đó:

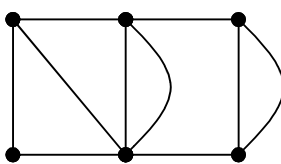
- $V \neq \emptyset$ là tập hợp hữu hạn gồm các **đỉnh** của đồ thị.
- E là **tập hợp** các cặp **không có thứ tự** gồm **hai phần tử khác nhau** của V gọi là các **cạnh**.

Như vậy, theo định nghĩa trên, trong một đơn đồ thị không thể có các cặp cạnh nối cùng một cặp đỉnh (do E là tập hợp nên không thể có 2 cặp trùng nhau), các cạnh đều không phân biệt thứ tự nên cạnh $[u,v]$ và cạnh $[v,u]$ đều được coi là một cạnh duy nhất, điều này phù hợp với việc biểu diễn các con đường 2 chiều, và hiển nhiên là không có cặp $[u,u]$ nào đó trong E .

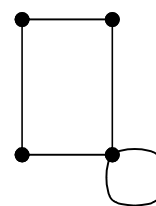
Ví dụ 1.1.



a. Đơn đồ thị vô hướng



b. Không phải đơn đồ thị vô hướng do có các cặp cạnh nối cùng một cặp đỉnh



c. Không phải đơn đồ thị vô hướng do có cạnh nối một đỉnh với chính nó.

Tuy nhiên, trên thực tế, cũng có thể trong một hệ thống giao thông vẫn tồn tại nhiều con đường đi nối cùng hai địa điểm, hoặc cũng có thể có một con đường để đi từ một địa điểm nào đó rồi lại quay về chính nó (đây có thể là một con đường nội bộ của một trung tâm mua sắm, ...). Khi đó, tính chất của đơn đồ thị vô hướng như định nghĩa trên không cho phép nó biểu diễn được hệ thống giao thông trong trường hợp này. Muốn vậy, ta phải dùng một loại đồ thị tổng quát hơn một chút: **đa đồ thị vô hướng**.

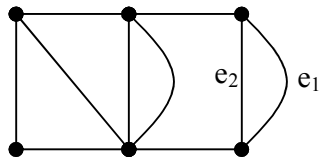
Định nghĩa 1.2. Đa đồ thị vô hướng là một bộ $G=\langle V,E \rangle$, trong đó

- $V \neq \emptyset$ là tập hợp hữu hạn gồm các **đỉnh** của đồ thị.
- E là một **họ** các cặp **không có thứ tự** của V gọi là các **cạnh**.

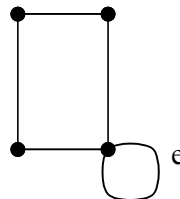
Lưu ý:

- Khi ta nói E là một họ nghĩa là nó có thể có những cặp trùng nhau (khác với khái niệm tập hợp).
- Các cạnh nối cùng một cặp đỉnh được gọi là các **cạnh song song**.

- Các cạnh nối từ một đỉnh với chính nó được gọi là **khuyên**.

Ví dụ 1.2.

- a. Đa đồ thị vô hướng. e_1 và e_2 là các cạnh song song.

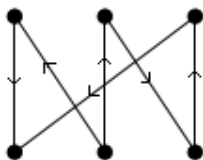


- b. Đa đồ thị vô hướng. e là khuyên

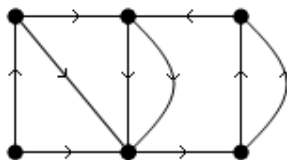
Điểm chung của hai loại đồ thị đã được định nghĩa ở trên là tính chất vô hướng (hai chiều) của các cạnh. Trong thực tế, cũng có khi ta phải chú trọng đến tính có hướng của các cạnh nối này (chẳng hạn như biểu diễn các con đường một chiều). Từ đó, ta có thêm loại đồ thị: **Đơn đồ thị có hướng** và **đa đồ thị có hướng**. Về cơ bản, hai loại này cũng tương tự như hai loại mà ta định nghĩa ở trên, chỉ thêm sự khác biệt là tính chất có thứ tự của các cạnh.

Định nghĩa 1.3. Đơn đồ thị có hướng là một bộ $G=\langle V, E \rangle$, trong đó:

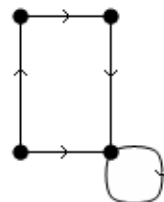
- $V \neq \emptyset$ là tập hợp hữu hạn gồm các **đỉnh** của đồ thị.
- E là **tập hợp** các cặp có thứ tự gồm **hai phần tử khác nhau** của V gọi là các **cung**.

Ví dụ 1.3.

- a. Đơn đồ thị vô hướng



- b. Không phải đơn đồ thị vô hướng do có các cặp cung nối cùng một cặp đỉnh

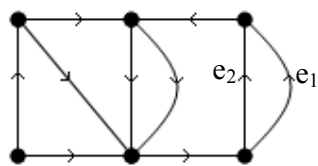


- c. Không phải đơn đồ thị vô hướng do có cung nối một đỉnh với chính nó.

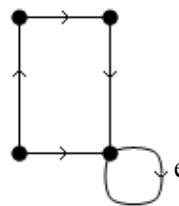
Định nghĩa 1.4. Đa đồ thị có hướng là một bộ $G=\langle V, E \rangle$, trong đó

- $V \neq \emptyset$ là tập hợp hữu hạn gồm các **đỉnh** của đồ thị.
- E là một **họ** các cặp có thứ tự của V gọi là các **cung**.

Các cung nối cùng một cặp đỉnh được gọi là các **cung song song**.

Ví dụ 1.4.

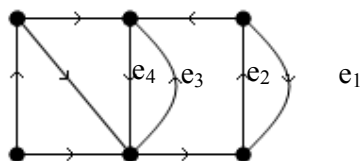
- a. Đa đồ thị có hướng. e_1 và e_2 là các cung song song.



- b. Đa đồ thị có hướng. e là khuyên

Chú ý:

- Đồ thị sau vẫn được coi là đơn đồ thị có hướng vì e_1 và e_2 , e_3 và e_4 không phải là 2 cung song song (do khác hướng).



- Một số tài liệu tách đa đồ thị thành 2 loại: đa đồ thị (chỉ có cạnh/cung song song mà không có khuyên) và giả đồ thị (có cạnh/cung song song và có cả khuyên). Tuy nhiên, để bớt phức tạp, chúng ta gộp cả hai loại này thành một và gọi tên chung là đa đồ thị.
- Đa đồ thị là dạng tổng quát hơn đơn đồ thị, nghĩa là một đơn đồ thị vẫn có thể được coi là đa đồ thị, nhưng ngược lại thì không đúng.
- Mặc dù tổng quát hơn nhưng đa đồ thị lại rất khó biểu diễn và xử lý trên máy tính. Chính vì thế trong phần lớn các ứng dụng người ta tìm cách biến các đa đồ thị bằng cách thêm một số đỉnh vào giữa các cạnh/cung song song hay các khuyên. Khi đó, đa đồ thị sẽ trở thành đơn đồ thị.
- Cũng chính vì lý do trên, các nội dung giới thiệu trong học phần này chủ yếu là trên đơn đồ thị. Để đơn giản, **chúng ta sẽ gọi là “đồ thị” thay cho “đơn đồ thị”**.

Phần tiếp theo sau đây sẽ giới thiệu cho chúng ta một số thuật ngữ cơ bản thường dùng trên đồ thị.

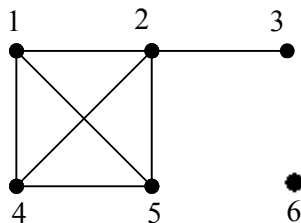
1.3 Một số thuật ngữ cơ bản

Định nghĩa 1.5. Cho đồ thị vô hướng $G=\langle V, E \rangle$.

- Hai đỉnh u và v của đồ thị được gọi là **kề nhau** nếu (u, v) là một cạnh của đồ thị.
- Nếu $e=(u, v)$ là cạnh của đồ thị thì ta nói cạnh này là **liên thuộc** với hai đỉnh u và v . Cạnh được nói là **nối** đỉnh u và v . Đỉnh u và v được gọi là **đỉnh đầu** của cạnh e .

Định nghĩa 1.6. Cho đồ thị vô hướng $G = \langle V, E \rangle$. Bậc của đỉnh v trong đồ thị, ký hiệu là $\deg(v)$, là số cạnh liên thuộc với nó. Đỉnh có bậc 0 được gọi là đỉnh cô lập, đỉnh có bậc 1 gọi là đỉnh treo.

Ví dụ 1.5. Cho đồ thị vô hướng $G = \langle V, E \rangle$ sau:



- $V = \{1, 2, 3, 4, 5, 6\}$
- $E = \{(1,2), (2,3), (1,4), (1,5), (2,4), (2,5), (4,5), (5,6)\}$
- Bậc của các đỉnh:

▪ $\deg(1) = 3$	$\deg(2) = 4$	$\deg(3) = 1$
▪ $\deg(4) = 3$	$\deg(5) = 3$	$\deg(6) = 0$
- Đỉnh 3 là đỉnh treo
- Đỉnh 6 là đỉnh cô lập

Định lý sau sẽ đề cập đến tính chất của bậc các đỉnh.

Định lý 1.1. Cho $G = \langle V, E \rangle$ là đồ thị vô hướng. Khi đó ta có tổng số bậc của các đỉnh của đồ thị sẽ bằng hai lần số cạnh của nó. Nói cách khác, ta có:

$$\sum_{v \in V} \deg(v) = 2 | E |$$

Việc chứng minh định lý này không khó. Ý tưởng chính của nó là trong quá trình xác định bậc của các đỉnh thì mỗi cạnh được đếm 2 lần.

Hệ quả 1.1. Trong đồ thị vô hướng, số đỉnh bậc lẻ là một số chẵn.

Chứng minh.

Theo định lý trên, tổng bậc của tất cả các đỉnh là một số chẵn ($2|E|$), do đó tổng bậc của các đỉnh bậc lẻ cũng là một số chẵn. Và do vậy, số đỉnh bậc lẻ phải là một số chẵn. ■

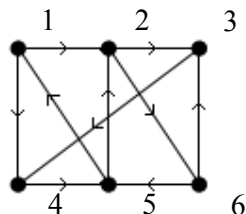
Định nghĩa 1.7. Cho đồ thị có hướng $G = \langle V, E \rangle$.

- Hai đỉnh u và v của đồ thị được gọi là **kề nhau** nếu (u, v) là một cung của đồ thị.
- Nếu $e = (u, v)$ là cung của đồ thị thì ta nói cung này **đi ra khỏi** đỉnh u vào **đi vào** đỉnh v . Đỉnh u được gọi là **đỉnh đầu** của cung e và đỉnh v được gọi là **đỉnh cuối** của cung e .

Định nghĩa 1.8. Cho đồ thị có hướng $G = \langle V, E \rangle$.

- **Bán bậc ra của đỉnh v** trong đồ thị, ký hiệu là $\deg^+(v)$, là số cạnh đi ra khỏi v .
- **Bán bậc vào của đỉnh v** trong đồ thị, ký hiệu là $\deg^-(v)$, là số cạnh vào v .

Ví dụ 1.6. Xét đồ thị có hướng $G = \langle V, E \rangle$ sau:



- $V = \{1, 2, 3, 4, 5, 6\}$
- $E = \{(1,2), (2,3), (1,4), (5,1), (5,2), (2,6), (6,3), (4,5), (6,5), (3,4)\}$
- Bậc của các đỉnh:

▪ Bán bậc ra:	$\deg^+(1)=2$	$\deg^+(2)=2$	$\deg^+(3)=1$
	$\deg^+(4)=1$	$\deg^+(5)=2$	$\deg^+(6)=2$
▪ Bán bậc vào:	$\deg^-(1)=1$	$\deg^-(2)=2$	$\deg^-(3)=2$
	$\deg^-(4)=2$	$\deg^-(5)=2$	$\deg^-(6)=1$

Tương tự như đồ thị vô hướng, đối với đồ thị có hướng ta cũng có kết quả gần tương tự về bậc của các đỉnh của đồ thị.

Định lý 1.2. Cho $G = \langle V, E \rangle$ là đồ thị có hướng. Tổng bán bậc ra của các đỉnh bằng tổng bán bậc vào của các đỉnh và bằng số cạnh của đồ thị.

$$\sum_{v \in V} \deg^+(v) = \sum_{v \in V} \deg^-(v) = |E|$$

Cách chứng minh và ý nghĩa định lý này cũng tương tự như định lý đối với đồ thị vô hướng.

Trong các ứng dụng về đồ thị, các bài toán về đường đi, tính liên thông chiếm vị trí rất lớn. Phần kế tiếp sẽ đề cập đến một số khái niệm mở đầu về các nội dung này.

1.4 Đường đi, chu trình và đồ thị liên thông

Định nghĩa 1.9. Cho đồ thị $G = \langle V, E \rangle$. Đường đi độ dài n từ đỉnh u đến đỉnh v (n là số nguyên dương) là dãy:

$$x_0, x_1, \dots, x_{n-1}, x_n$$

trong đó $u = x_0, v = x_n, (x_i, x_{i+1}) \in E, i = 0, 1, \dots, n-1$.

* Ta sẽ dùng thuật ngữ đồ thị để chỉ chung cho cả đồ thị vô hướng và đồ thị có hướng

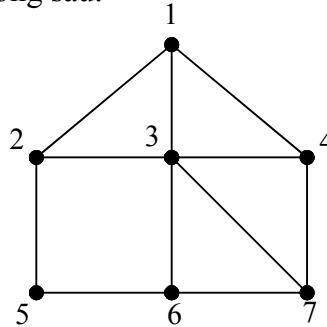
Đường đi nói trên còn có thể được biểu diễn bằng dãy các cạnh/cung:

$$(x_0, x_1), (x_1, x_2), \dots, (x_{n-1}, x_n)$$

Đỉnh u gọi là đỉnh đầu của đường đi, đỉnh v gọi là đỉnh cuối của đường đi.

Đường đi có đỉnh đầu và đỉnh cuối trùng nhau ($u=v$) gọi là chu trình.

Ví dụ 1.7. Cho đồ thị vô hướng sau:



Một số đường đi từ đỉnh 2 đến đỉnh 7:

- Đường đi d_1 : 2 3 4 7 (đường đi độ dài 3)
- Đường đi d_2 : 2 3 4 1 3 6 7 (đường đi độ dài 6)
- Đường đi d_3 : 2 3 4 1 3 4 7 (đường đi độ dài 6)

Một số chu trình trên đồ thị trên:

- Chu trình C_1 : 1 2 3 1 (chu trình có độ dài 3)
- Chu trình C_2 : 1 2 3 7 6 3 4 1 (chu trình có độ dài 7)
- Chu trình C_3 : 1 3 4 7 3 4 1 (chu trình có độ dài 6)

Định nghĩa 1.10. Cho đồ thị $G = \langle V, E \rangle$.

- Đường đi hay chu trình trên G được gọi là **đơn** nếu như không có cạnh nào bị lặp lại trên đường đi.
- Đường đi hay chu trình trên G được gọi là **sơ cấp** nếu như không có đỉnh nào bị lặp lại trên đường đi.

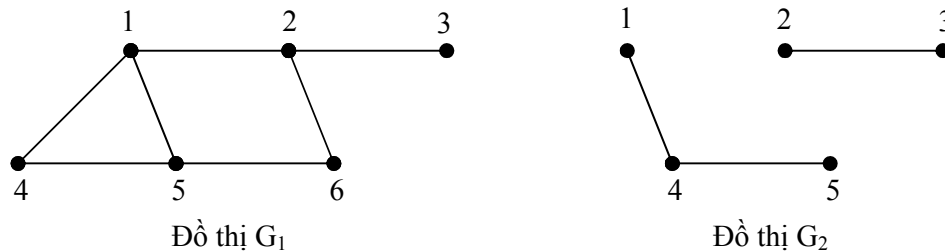
Ví dụ 1.8. Xét các đường đi và chu trình ở ví dụ trên, ta thấy:

- Đường đi d_1 là đường đi sơ cấp (cũng là đường đi đơn).
- Đường đi d_2 là đường đi đơn (chỉ bị lặp đỉnh 3, nhưng không lặp cạnh).
- Đường đi d_3 không là đường đi đơn (cũng không là đường đi sơ cấp) do có lặp lại cạnh (3,4).
- Chu trình C_1 là chu trình sơ cấp (cũng là chu trình đơn).
- Chu trình C_2 là chu trình đơn (chỉ bị lặp lại đỉnh 3, nhưng không lặp cạnh).
- Chu trình C_3 không là chu trình đơn (cũng không là chu trình sơ cấp) do có lặp lại cạnh (3,4).

Khi dùng đồ thị để biểu diễn một hệ thống nào đó, chẳng hạn hệ thống các máy tính kết nối với nhau, một trong những điều ta quan tâm nhất là liệu hai máy tính nào đó có thể được nối với nhau hay không? Đây là tính chất liên thông của một mạng, nếu tính liên thông không được đảm bảo thì mạng máy tính sẽ không thể hoạt động được.

Định nghĩa 1.11. Đồ thị vô hướng $G = \langle V, E \rangle$ được gọi là **liên thông** nếu luôn tìm được đường đi giữa hai đỉnh bất kỳ của nó.

Ví dụ 1.9. Xét các đồ thị vô hướng sau:



Trong 2 đồ thị trên thì G_1 là đồ thị liên thông, còn G_2 không phải là đồ thị liên thông vì giữa hai đỉnh 1 và 2 không tồn tại một đường đi nào.

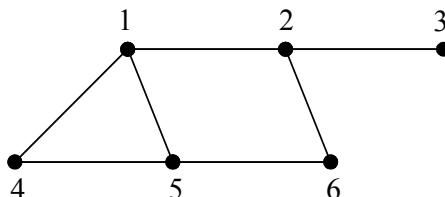
Định nghĩa 1.12. Cho đồ thị $G = (V, E)$. Đồ thị $H = \langle W, F \rangle$ được gọi là **đồ thị con** của G nếu và chỉ nếu $W \subseteq V$ và $F \subseteq E$.

Trong trường hợp một đồ thị vô hướng G không liên thông, nó sẽ được phân thành các đồ thị con độc lập nhau và chúng đều liên thông. Mỗi đồ thị con như vậy được gọi là một **thành phần liên thông** của G .

Ví dụ 1.10. Đồ thị G_2 trong ví dụ trên là đồ thị có 2 thành phần liên thông. Thành phần liên thông thứ nhất gồm 3 đỉnh: 1, 4, và 5. Thành phần liên thông thứ hai gồm hai đỉnh: 2 và 3.

Định nghĩa 1.13. Cho đồ thị vô hướng $G = \langle V, E \rangle$. Đỉnh v của đồ thị được gọi là **đỉnh rẽ nhánh** nếu việc loại bỏ v và các cạnh liên thuộc với nó ra khỏi đồ thị sẽ làm tăng số thành phần liên thông của đồ thị. Cạnh e của đồ thị được gọi là **cầu** nếu việc loại bỏ nó ra khỏi đồ thị sẽ làm tăng số thành phần liên thông của đồ thị.

Ví dụ 1.11. Xét đồ thị sau:



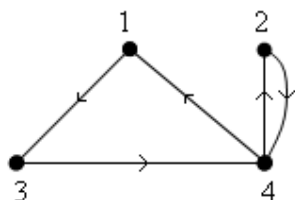
Trong đồ thị trên, đỉnh 2 là đỉnh rẽ nhánh vì việc loại đỉnh này cùng với các cạnh (2,3), (2,1), (2,6) sẽ làm đồ thị có 2 thành phần liên thông. Cạnh (2,3) là cầu. Các cạnh còn lại đều không phải là cầu.

Đối với đồ thị có hướng khái niệm liên thông khó thỏa mãn hơn do các cung bị hạn chế về chiều. Từ đó, bên cạnh khái niệm liên thông như đề cập trong đồ thị vô hướng, ta sẽ đưa thêm khái niệm liên thông nhẹ hơn: liên thông yếu.

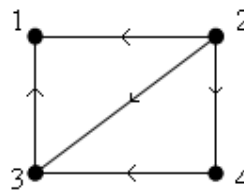
Định nghĩa 1.14. Cho $G = \langle V, E \rangle$ là đồ thị có hướng.

- G được gọi là **liên thông mạnh** nếu luôn tìm được đường đi giữa hai đỉnh bất kỳ của nó.
- G được gọi là **liên thông yếu** nếu đồ thị vô hướng tương ứng với nó (đồ thị vô hướng có được bằng cách biến các cung một chiều thành các cạnh hai chiều) là đồ thị vô hướng liên thông.

Ví dụ 1.12. Xét các đồ thị có hướng sau:



Đồ thị G_1



Đồ thị G_2

- Đồ thị G_1 là đồ thị liên thông mạnh.
- Đồ thị G_2 không là đồ thị liên thông mạnh và từ đỉnh 1 đến đỉnh 2 không tồn tại đường đi nào. G_2 là đồ thị liên thông yếu vì nếu biến các cung có hướng thành các cạnh vô hướng thì nó là đồ thị liên thông.

1.5 Biểu diễn đồ thị trên máy tính

Lý thuyết đồ thị được ứng dụng trong rất nhiều lĩnh vực khác nhau. Để sử dụng được đồ thị hiệu quả và nhanh chóng hơn, chúng ta phải biểu diễn và xử lý được đồ thị với máy tính. Cách biểu diễn thông thường bằng hình vẽ và mô tả tập hợp sẽ không phù hợp với cách thức lưu trữ dữ liệu và xử lý trên máy tính. Chúng ta phải tìm một cấu trúc dữ liệu thích hợp để biểu diễn đồ thị trên máy tính.

Có nhiều phương pháp khác nhau để biểu diễn đồ thị trên máy tính. Sau đây chúng ta sẽ lần lượt tìm hiểu một số phương pháp thông dụng.

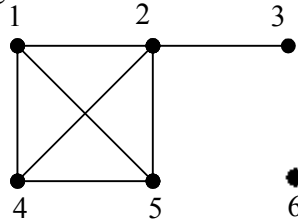
1.5.1 Biểu diễn đồ thị bằng ma trận kề

Định nghĩa 1.15. Cho đồ thị $G = \langle V, E \rangle$, với tập đỉnh $V = \{v_1, v_2, \dots, v_n\}$. Ta gọi ma trận kề của đồ thị là ma trận A , kích thước $n \times n$ được xác định như sau:

$$A_{ij} = \begin{cases} 1, & (v_i, v_j) \in E \\ 0, & (v_i, v_j) \notin E \end{cases}$$

Ví dụ 1.13.

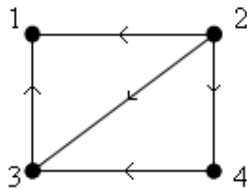
a. Xét đồ thị vô hướng sau:



Ma trận kề của đồ thị trên sẽ là:

$$A = \begin{matrix} & \begin{matrix} 1 & 2 & 3 & 4 & 5 & 6 \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \\ 5 \\ 6 \end{matrix} & \begin{bmatrix} 0 & 1 & 0 & 1 & 1 & 0 \\ 1 & 0 & 1 & 1 & 1 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 & 1 & 0 \\ 1 & 1 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix} \end{matrix}$$

b. Xét đồ thị có hướng sau:



Ma trận kề của đồ thị trên sẽ là:

$$A = \begin{matrix} & \begin{matrix} 1 & 2 & 3 & 4 \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \end{matrix} & \begin{bmatrix} 0 & 1 & 0 & 0 \\ 1 & 0 & 1 & 1 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 \end{bmatrix} \end{matrix}$$

Nhận xét.

- Chúng ta có thể nhận thấy rằng, ma trận kề của đồ thị vô hướng luôn luôn là ma trận đối xứng. Còn ma trận của đồ thị có hướng thì có thể không có tính chất này.

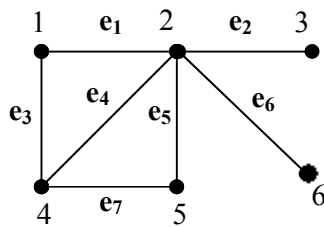
- Đối với đồ thị vô hướng, tổng các phần tử trên dòng i (hay trên cột i) sẽ là bậc của đỉnh v_i của đồ thị
- Đối với đồ thị có hướng, tổng các phần tử trên dòng i (tương ứng, trên cột i) sẽ là bán bậc ra (bán bậc vào) của đỉnh v_i của đồ thị.

1.5.2 Ma trận liên thuộc đỉnh – cạnh

Định nghĩa 1.16. Cho $G = \langle V, E \rangle$ là đồ thị vô hướng với tập đỉnh $V = \{v_1, v_2, \dots, v_n\}$ và tập cạnh $E = \{e_1, e_2, \dots, e_m\}$. Ma trận liên thuộc đỉnh cạnh biểu diễn đồ thị G là ma trận có kích thước $n \times m$ được xác định như sau:

$$A_{ij} = \begin{cases} 1, & \text{nếu } v_i \text{ là đỉnh đầu của cạnh } e_j \\ 0, & \text{nếu } v_i \text{ không là đỉnh đầu của cạnh } e_j \end{cases}$$

Ví dụ 1.14. Ma trận liên thuộc đỉnh-cạnh của đồ thị dưới đây là:

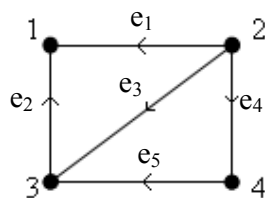


$$A = \begin{matrix} & \begin{matrix} e_1 & e_2 & e_3 & e_4 & e_5 & e_6 & e_7 \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \\ 5 \\ 6 \end{matrix} & \begin{bmatrix} 1 & 0 & 1 & 0 & 0 & 0 & 0 \\ 1 & 1 & 0 & 1 & 1 & 1 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 1 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 \end{bmatrix} \end{matrix}$$

Định nghĩa 1.17. Cho $G = \langle V, E \rangle$ là đồ thị có hướng với tập đỉnh $V = \{v_1, v_2, \dots, v_n\}$ và tập các cung $E = \{e_1, e_2, \dots, e_m\}$. Ma trận liên thuộc đỉnh-cạnh biểu diễn đồ thị G là ma trận có kích thước $n \times m$ được xác định như sau:

$$A_{ij} = \begin{cases} 1, & \text{nếu } v_i \text{ là đỉnh đầu của cung } e_j \text{ (cung } e_j \text{ đi ra từ } v_i) \\ -1, & \text{nếu } v_i \text{ là đỉnh cuối của cung } e_j \text{ (cung } e_j \text{ đi vào } v_i) \\ 0, & \text{nếu } v_i \text{ không là đỉnh đầu mút nào của cung } e_j \end{cases}$$

Ví dụ 1.15. Ma trận liên thuộc đỉnh-cạnh của đồ thị vô hướng dưới đây là:



$$A = \begin{matrix} & \begin{matrix} e_1 & e_2 & e_3 & e_4 & e_5 \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \end{matrix} & \begin{bmatrix} -1 & -1 & 0 & 0 & 0 \\ 1 & 0 & 1 & 1 & 0 \\ 0 & 1 & -1 & 0 & -1 \\ 0 & 0 & 0 & -1 & 1 \end{bmatrix} \end{matrix}$$

Nhận xét.

- Ma trận liên thuộc đỉnh-cạnh sẽ tiết kiệm bộ nhớ hơn khi đồ thị có ít cạnh/cung.
- Trong ma trận của đồ thị vô hướng, số số 1 trên dòng i sẽ là bậc của đỉnh v_i .
- Trong ma trận của đồ thị có hướng, số số 1 trên dòng i sẽ là bán bậc ra của đỉnh v_i , còn số số -1 trên dòng i sẽ là bán bậc vào của đỉnh v_i .

Ngoài hai phương pháp biểu diễn đồ thị đã trình bày, còn một số cách khác như dùng danh sách cạnh/cung hoặc sử dụng danh sách kề, ... (xin xem thêm chi tiết trong tài liệu tham khảo).

Chương 2.

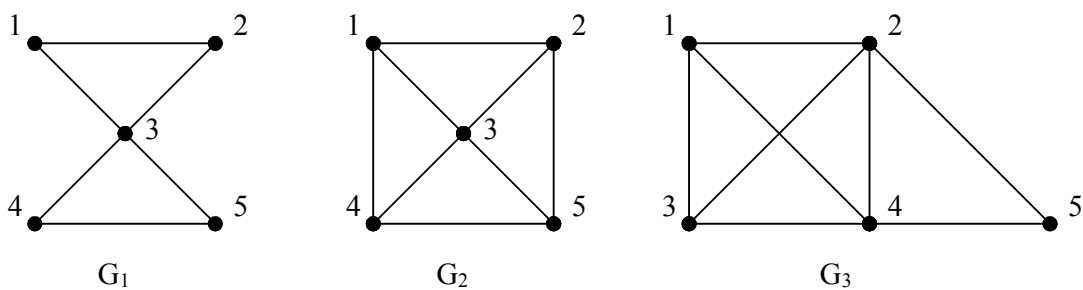
Đồ thị Euler và đồ thị Hamilton

2.1 Đồ thị Euler

Định nghĩa 2.1. Cho đồ thị $G = \langle V, E \rangle$. Chu trình đơn trong G đi qua tất cả các cạnh của nó được gọi là chu trình Euler. Đường đi đơn đi qua tất cả các cạnh của đồ thị được gọi là đường đi Euler.

Đồ thị G được gọi là đồ thị Euler nếu nó có chứa chu trình Euler. G được gọi là đồ thị nửa Euler nếu nó có chứa đường đi Euler.

Ví dụ 2.1. Xét các đồ thị dưới đây:



- Đồ thị G_1 là đồ thị Euler (hiển nhiên cũng là nửa Euler) vì nó có chứa chu trình Euler: 1 2 3 5 4 3 1
- Đồ thị G_2 không là đồ thị Euler cũng không phải là đồ thị nửa Euler.
- Đồ thị G_3 dù không có chứa chu trình Euler nhưng nó là đồ thị nửa Euler do có chứa đường đi Euler: 1 2 3 1 4 2 5 4 3.

Rõ ràng đồ thị Euler có một tính chất rất hay là chúng ta có thể đi qua tất cả các cạnh của đồ thị, mỗi cạnh chỉ đi qua một lần và quay trở về đỉnh ban đầu. Đồ thị Euler có rất nhiều ứng dụng như: giải bài toán 7 cái cầu ở thành phố Königsberg, bài toán vẽ hình bằng 1 nét, ... Để giải được các bài toán này, ta phải mô hình chúng thành đồ thị và xác định xem liệu đó có phải là đồ thị Euler hay không. Câu hỏi đặt ra là “*khi nào thì một đồ thị là Euler?*”. Thật may mắn câu trả lời trọn vẹn đã được Euler tìm ra vào năm 1736 thể hiện qua định lý dưới đây:

Định lý 2.1. (Định lý Euler) Đồ thị vô hướng liên thông G là đồ thị Euler khi và chỉ khi mọi đỉnh của G đều có bậc chẵn.

Để chứng minh định lý này, trước hết ta sẽ chứng minh bổ đề sau:

Bổ đề. Nếu đồ thị vô hướng G có bậc các đỉnh không nhỏ hơn 2 thì G có chứa ít nhất một chu trình đơn.

Chứng minh bổ đề.

Gọi v là một đỉnh nào đó. Xuất phát từ v , ta bắt đầu đi qua các đỉnh khác theo các cạnh của đồ thị. Do bậc của các đỉnh đều lớn hơn hay bằng 2 nên tại mỗi đỉnh ta đều có thể đi vào bằng một đường và đi ra bằng một đường khác. Do số đỉnh của đồ thị là hữu hạn nên đến một lúc nào đó, ta sẽ phải đi lại 1 đỉnh đã đi trước đó, và như vậy thì ta đã có một chu trình đơn trong đồ thị. ■

Sau đây ta sẽ chứng minh định lý.

Chứng minh định lý.

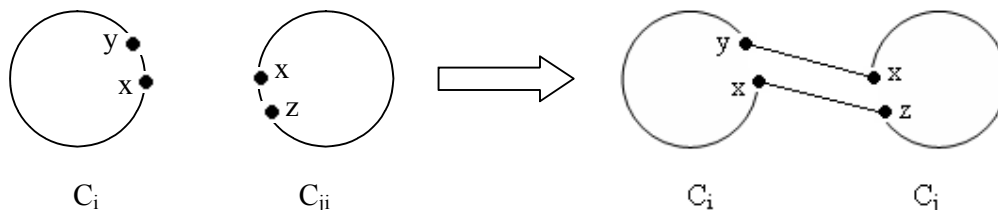
Cần. Nếu G là đồ thị Euler thì các đỉnh trong G đều có bậc chẵn.

Giả sử G là đồ thị Euler, khi đó tồn tại chu trình Euler C trong G . Như vậy, cứ mỗi lần chu trình này đi qua một đỉnh thì bậc của đỉnh này sẽ tăng lên 2. Mặt khác, do chu trình C đi qua tất cả các đỉnh nên trong quá trình đi như trên sẽ không còn sót cạnh nào. Vậy các đỉnh của đồ thị đều phải có bậc chẵn.

Đủ. Nếu các đỉnh trong G đều có bậc chẵn thì G là đồ thị Euler.

Ta phải chứng minh G có chứa 1 chu trình Euler. Việc chứng minh phần này cũng chính là cách tìm chu trình Euler của G .

Trước hết, do G liên thông và các đỉnh đều là bậc chẵn nên các đỉnh của G đều có bậc lớn hơn hay bằng 2. Do đó trong G phải có chứa một chu trình đơn C_1 . Nếu C_1 đã đi qua hết tất cả các cạnh của G thì C_1 chính là chu trình Euler cần tìm. Nếu không, thì loại bỏ các cạnh đã đi qua (trong C_1) ra khỏi đồ thị, ta được đồ thị H với số bậc các đỉnh cũng đều là bậc chẵn. Trong mỗi thành phần liên thông của H , làm tương tự ta cũng thu được các chu trình đơn $C_2, C_3, \dots$. Quá trình cứ tiếp tục cho đến khi tất cả các đỉnh được đi qua. Giả sử cuối cùng, ta có được k chu trình đơn: $C_1, C_2, \dots, C_k$. Ta sẽ tìm cách kết nối chúng thành một chu trình đơn duy nhất. Trong số các chu trình trên, có ít nhất 2 chu trình có chung một đỉnh (vì nếu không, G không phải là đồ thị liên thông). Giả sử C_i và C_j là 2 chu trình có điểm chung, gọi điểm chung đó là x . Ta sẽ “mở” 2 chu trình này ra tại x và nối chúng lại thành một chu trình đơn lớn hơn theo hình vẽ dưới đây:



Theo hình trên, ta thấy rằng quá trình kết nối không bỏ mất cạnh nào, và chu trình thu được cũng là một chu trình đơn.

Tiếp tục làm như trên (tìm 2 chu trình có điểm chung, kết nối thành chu trình lớn hơn) cho đến khi ta được một chu trình đơn duy nhất. Đó chính là chu trình Euler cần tìm. Vậy G là đồ thị Euler. ■

Định lý trên cho ta một công cụ hữu hiệu để xác định một đồ thị có là Euler hay không: chỉ việc đếm bậc của các đỉnh.

Hệ quả sau đây sẽ đề cập đến việc nhận biết đồ thị nửa Euler.

Hệ quả 2.1. Đồ thị vô hướng liên thông G là nửa Euler khi và chỉ khi nó có không quá đỉnh bậc lẻ.

Chứng minh. Thật vậy G chỉ có thể có 0 đỉnh bậc lẻ hoặc 2 đỉnh bậc lẻ. Nếu G không có đỉnh bậc lẻ nào thì G là đồ thị Euler và do đó hiển nhiên nó là đồ thị nửa Euler. Nếu G có đúng 2 đỉnh bậc lẻ, khi đó, tưởng tượng rằng ta sẽ thêm vào 1 cạnh ảo nối hai đỉnh này. Và như vậy thì tất cả các đỉnh sẽ đều có bậc chẵn và ta có thể tìm được chu trình Euler theo định lý 2.1. Bây giờ loại bỏ cạnh ảo ra khỏi chu trình, ta sẽ có đường đi Euler của đồ thị ban đầu. ■

Nhận xét. Theo cách chứng minh của hệ quả trên, ta thấy rằng đường đi Euler trong đồ thị nửa Euler phải luôn luôn bắt đầu và kết thúc từ các đỉnh bậc lẻ.

Tương tự như đồ thị vô hướng, tư tưởng của định lý Euler vẫn đúng cho đồ thị có hướng. Chỉ có điều, ta phải điều chỉnh một chút các điều kiện cho phù hợp với đồ thị có hướng.

Định lý 2.2. Đồ thị có hướng liên thông mạnh là đồ thị Euler khi và chỉ khi mọi đỉnh của nó đều có bán bậc ra bằng bán bậc vào. Nghĩa là:

$$\forall v \in V, \deg^+(v) = \deg^-(v)$$

2.2 Đồ thị Hamilton

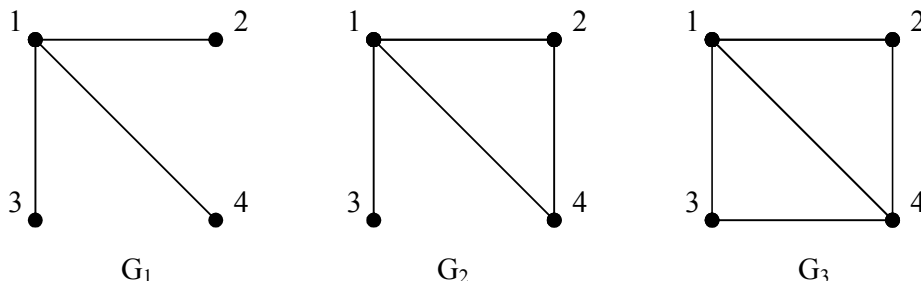
Trong phần trên, chúng ta đã khảo sát một loại đồ thị đặc biệt: đồ thị mà ta có thể đi qua tất cả các cạnh của nó, mỗi cạnh một lần. Cũng tương tự ý tưởng như vậy, trong phần này, chúng ta sẽ xét các đồ thị mà ta có thể đi qua tất cả các đỉnh của nó, mỗi đỉnh một lần.

Định nghĩa 2.2. Cho đồ thị $G = \langle V, E \rangle$. Chu trình sơ cấp[†] trong G đi qua tất cả các đỉnh của nó được gọi là chu trình Hamilton. Đường đi sơ cấp đi qua tất cả các đỉnh của đồ thị được gọi là đường đi Hamilton.

Đồ thị G được gọi là đồ thị Hamilton nếu nó có chứa chu trình Hamilton. G được gọi là đồ thị nửa Hamilton nếu nó có chứa đường đi Hamilton.

Dễ thấy rằng đồ thị Hamilton cũng là đồ thị nửa Hamilton, ngược lại thì không luôn đúng.

Ví dụ 2.2. Xét các đồ thị dưới đây:



- G_1 không là đồ thị Hamilton cũng không là đồ thị nửa Hamilton.
- G_2 là đồ thị nửa Hamilton vì có chứa đường đi Hamilton: 3 1 2 4. Nhưng G_2 không là đồ thị Hamilton.
- G_3 là đồ thị Hamilton vì có chứa chu trình Hamilton: 1 2 4 3

Trong phần trên, khi khảo sát đồ thị Euler, chúng ta được giới thiệu một định lý rất hiệu quả và đầy đủ. Nó thể hiện được điều kiện cần và đủ (hai chiều) để một đồ thị liên thông là đồ thị Euler. Và như vậy, lớp các đồ thị Euler đã được xác định rất rõ ràng. Rất tiếc, đối với việc nhận dạng đồ thị Hamilton, cho đến bây giờ chúng ta vẫn chưa có được một kết quả hoàn hảo như vậy. Phần lớn các kết quả nghiên cứu cho đến thời điểm này chủ yếu cung cấp điều kiện đủ để một đồ thị là Hamilton.

Sau đây chúng ta giới thiệu một số định lý về điều kiện đủ để một đồ thị là đồ thị Hamilton.

Định lý 2.3. (Dirak, 1952). Nếu đồ thị vô hướng G với n đỉnh ($n > 2$), mỗi đỉnh có bậc không nhỏ hơn $n/2$ thì G là đồ thị Hamilton.

Định lý 2.4. (Dirak, tổng quát) Nếu G là đồ thị có hướng liên thông mạnh với n đỉnh. Nếu các đỉnh đều có bán bậc ra và bán bậc vào không nhỏ hơn $n/2$ thì G là đồ thị Hamilton.

Nghiên cứu đồ thị Hamilton và các tính chất của nó vẫn là một hướng mở hiện nay. Các nhà nghiên cứu vẫn đang tìm kiếm một công cụ hữu hiệu để nhận biết các đồ

[†] Không lặp lại đỉnh

thị Hamilton cũng như xác định đầy đủ lớp các đồ thị Hamilton. Đây thực sự là một thách thức trong lĩnh vực lý thuyết đồ thị.

Chương 3.

Đồ thị có trọng số và bài toán đường đi ngắn nhất

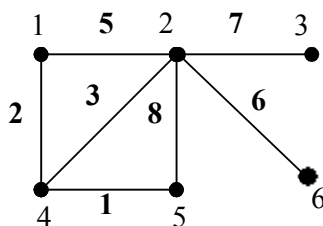
3.1 Đồ thị có trọng số

Trong các phần vừa qua, chúng ta đã khảo sát các khái niệm cơ bản trên đồ thị. Cho đến thời điểm này, chúng ta đã nói đến đồ thị như là một bộ gồm có tập các đỉnh và tập các cạnh nối các đỉnh với nhau. Trên thực tế trong một số các ứng dụng, việc sử dụng đồ thị với các đỉnh và các cạnh để biểu diễn là chưa đủ. Chẳng hạn như đối với bài toán giao thông, người ta không chỉ quan tâm một địa điểm này có nối với một địa điểm khác hay không mà người ta còn quan tâm đến cả đường nối ấy xa hay gần, khoảng cách bao nhiêu. Rõ ràng, trong các hệ thống giao thông, việc đưa thêm được yếu tố khoảng cách vào đồ thị có ý nghĩa rất lớn, nó làm tăng thêm khả năng ứng dụng của lý thuyết đồ thị.

Trong phần này, chúng ta sẽ tìm hiểu loại đồ thị có khả năng đáp ứng yêu cầu trên: **đồ thị có trọng số**.

Định nghĩa 3.1. Đồ thị có trọng số là đồ thị mà mỗi cạnh (hay cung) của nó được gán thêm một số thực, gọi là trọng số của cạnh (cung), thể hiện chi phí phải tốn (khoảng cách, thời gian, tiền bạc, ...) khi đi qua cạnh (cung) đó.

Ví dụ 3.1. Xét một mô hình kết nối điện thoại được mô phỏng trên đồ thị có trọng số dưới đây: các đỉnh tương ứng với các trạm điện thoại, các cạnh tương ứng với đường dây kết nối giữa các trạm và trọng số các cạnh thể hiện chi phí phải tốn khi thực hiện một kết nối liên lạc giữa hai trạm.



Để biểu diễn đồ thị có trọng số trên máy tính, ta dùng ma trận kề trọng số. Về cơ bản, ma trận kề trọng số của đồ thị cũng gần giống với ma trận kề, chỉ có điều, trong khi ma trận kề chỉ gồm các số 0 hay 1 thì ma trận kề trọng số sẽ bao gồm trọng số của các đỉnh, cũng như các giá trị vô cực, ...

Định nghĩa 3.2. Ma trận kề trọng số của đồ thị có trọng số $G = \langle V, E \rangle$, $V = \{v_1, v_2, \dots, v_n\}$, là một ma trận kích thước $n \times n$, được xác định như sau:

$$A_{ij} = \begin{cases} \text{Trọng số của cạnh } (v_i, v_j), & \text{nếu } (v_i, v_j) \in E \\ \infty & \text{nếu } (v_i, v_j) \notin E \end{cases}$$

Ví dụ 3.2. Ma trận kề trọng số của đồ thị trong ví dụ 3.1 ở trên như sau:

$$A = \begin{bmatrix} \infty & 5 & \infty & 2 & \infty & \infty \\ 5 & \infty & 7 & 3 & 8 & 6 \\ \infty & 7 & \infty & \infty & \infty & \infty \\ 2 & 3 & \infty & \infty & 1 & \infty \\ \infty & 8 & \infty & 1 & \infty & \infty \\ \infty & 6 & \infty & \infty & \infty & \infty \end{bmatrix}$$

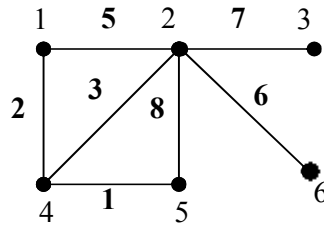
Chú ý: Ta thường dùng ký hiệu $a[v_i, v_j]$ để chỉ trọng số của cạnh (v_i, v_j) . Trong trường hợp (v_i, v_j) không là cạnh của đồ thị thì $a[v_i, v_j] = \infty$. Điều này cũng hoàn toàn tự nhiên vì một khi hai đỉnh không được nối với nhau thì chi phí để đi từ đỉnh này đến đỉnh kia là vô cùng lớn (lớn đến mức, không thể đi được).

3.2 Bài toán đường đi ngắn nhất

Trong các ứng dụng thực tế, bài toán tìm đường đi ngắn nhất giữa hai đỉnh của một đồ thị liên thông có một ý nghĩa to lớn. Có thể dẫn về bài toán như vậy nhiều bài toán thực tế quan trọng. Ví dụ, bài toán chọn một hành trình tiết kiệm nhất (theo tiêu chuẩn hoặc khoảng cách hoặc thời gian hoặc chi phí) trên một mạng giao thông đường bộ, đường thủy hoặc đường không; bài toán chọn một phương pháp tiết kiệm nhất để đưa ra một hệ thống động lực từ trạng thái xuất phát đến trạng thái đích, bài toán lập lịch thi công các công đoạn trong một công trình thi công lớn, bài toán lựa chọn đường truyền tin với chi phí nhỏ nhất trong mạng thông tin, v.v... Hiện nay có rất nhiều phương pháp để giải các bài toán như vậy. Thế nhưng, thông thường, các thuật toán được xây dựng dựa trên cơ sở lý thuyết đồ thị tỏ ra là các thuật toán có hiệu quả cao nhất. Trong phần này chúng ta sẽ xét bài toán đường đi ngắn nhất và một số thuật toán để giải bài toán này.

Định nghĩa 3.3. Cho G là đồ thị có trọng số và (P) là một đường đi trên G . Ta định nghĩa độ dài của đường đi (P) là tổng trọng số của tất cả các cạnh trên (P) .

Ví dụ 3.3. Xét đồ thị có trọng số dưới đây:



Độ dài của đường đi (P_1): 1 2 5 4 2 3 là: $5 + 8 + 1 + 3 + 7 = 24$.

Độ dài của đường đi (P_2): 1 4 2 6 là: $2 + 1 + 6 = 9$.

Bài toán tìm đường đi ngắn nhất trên đồ thị dưới dạng tổng quát có thể phát biểu như sau: **tìm đường đi có độ dài nhỏ nhất từ một đỉnh xuất phát $s \in V$ đến đỉnh cuối (đích) $t \in V$.**

Đường đi như vậy ta sẽ gọi là đường đi ngắn nhất từ s đến t còn độ dài của nó ta sẽ ký hiệu là $d(s,t)$ và còn gọi là khoảng cách từ s đến t (khoảng cách định nghĩa như vậy có thể là số âm). Nếu như không tồn tại đường đi từ s đến t thì ta sẽ đặt $d(s,t) = \infty$. Rõ ràng, nếu như mỗi chu trình trong đồ thị đều có độ dài dương, trong đường đi ngắn nhất không có đỉnh nào bị lặp lại (đường đi sơ cấp). Mặt khác nếu trong đồ thị có chu trình với độ dài âm (chu trình như vậy để gọi ngắn gọn ta gọi là chu trình âm) thì khoảng cách giữa một số cặp đỉnh nào đó của đồ thị có thể là không xác định, bởi vì, bằng cách đi vòng theo chu trình này một số đủ lớn lần, ta có thể chỉ ra đường đi giữa các đỉnh này có độ dài nhỏ hơn bất cứ số thực cho trước nào. Chính vì thế, để bài toán tìm đường đi ngắn nhất có lời giải, ta phải giả thiết: **đồ thị không chứa chu trình có độ dài âm.**

3.2.1 Đường đi ngắn nhất xuất phát từ một đỉnh.

Trong phần này, chúng ta sẽ xét thuật toán để tìm đường đi ngắn nhất từ một đỉnh $s \in V$ cho trước đến tất cả các đỉnh còn lại của đồ thị. Để biểu diễn lời giải của bài toán này, ta sẽ sử dụng hai mảng:

- Mảng d để lưu khoảng cách ngắn nhất từ s đến các đỉnh, chẳng hạn như $d[v]$ là độ dài đường đi ngắn nhất từ s đến v .
- Mảng $Truoc$ để lưu đỉnh trước của các đỉnh trên đường đi ngắn nhất ở trên, ví dụ, $Truoc[v]$ sẽ là đỉnh nằm trước v trên đường đi ngắn nhất từ s đến v . Mảng này dùng để lần lại kết quả khi kết thúc thuật toán (giống khái niệm bảng lưu vết trong quy hoạch động).

Phần lớn các thuật toán tìm khoảng cách giữa hai đỉnh s và t được xây dựng nhờ kỹ thuật tính toán mà ta có thể mô tả đại thể như sau: từ ma trận trọng số $a[u,v]$, $u,v \in$

V, ta tính cận trên $d[v]$ của khoảng cách từ s đến tất cả các đỉnh $v \in V$. Mỗi khi phát hiện

$$d[u] + a[u,v] < d[v] \quad (1)$$

cận trên $d[v]$ sẽ được làm tốt lên: $d[v] + a[u,v]$.

Quá trình đó sẽ kết thúc khi nào chúng ta không làm tốt thêm được bất kỳ cận trên nào. Khi đó, rõ ràng giá trị của mỗi $d[v]$ sẽ cho khoảng cách từ đỉnh s đến đỉnh v . Khi thể hiện kỹ thuật tính toán này trên máy tính, cận trên $d[v]$ sẽ được gọi là nhãn của đỉnh v , còn việc tính lại các cận này sẽ được gọi là thủ tục gán. Nhận thấy rằng để tính khoảng cách từ s đến t , ở đây, ta phải tính khoảng cách từ s đến tất cả các đỉnh còn lại của đồ thị. Hiện nay vẫn chưa biết thuật toán nào cho phép tìm đường đi ngắn nhất giữa hai đỉnh làm việc thực sự hiệu quả hơn những thuật toán tìm đường đi ngắn nhất từ một đỉnh đến tất cả các đỉnh còn lại.

Sơ đồ tính toán mà ta vừa mô tả còn chưa xác định, bởi vì còn phải chỉ ra thứ tự các đỉnh u và v để kiểm tra điều kiện (1). Thứ tự chọn này có ảnh hưởng rất lớn đến hiệu quả của thuật toán.

Bây giờ ta sẽ mô tả thuật toán Ford-Bellman tìm đường đi ngắn nhất từ đỉnh s đến tất cả các đỉnh còn lại của đồ thị. Thuật toán làm việc trong trường hợp trọng số của các cung là tùy ý, nhưng giả thiết rằng trong đồ thị không có chu trình âm.

Thuật toán Ford-Bellman:

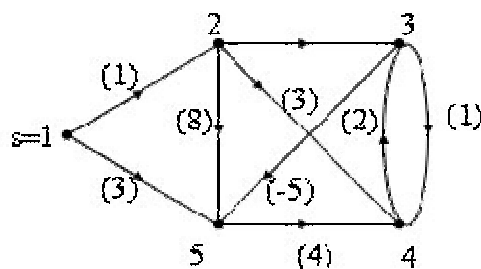
```

Begin
  (* Khởi tạo *)
  for  $v \in V$  do
    Begin
       $d[v] := a[s,v];$ 
       $Truoc[v] := s;$ 
    End;
  (* Bắt đầu *)
   $d[s] := 0;$ 
  for  $k := 1$  to  $n-2$  do
    for  $v \in V \setminus \{s\}$  do
      for  $u \in V$  do
        if  $d[v] > d[u] + a[u,v]$  then
          Begin
             $d[v] := d[u] + a[u,v];$ 
             $Truoc[v] := u;$ 
          End;
      End;
  End;

```

Tính đúng đắn của thuật toán có thể chứng minh trên cơ sở trên nguyên lý tối ưu của quy hoạch động. Rõ ràng là độ phức tạp tính toán của thuật toán là $O(n^3)$. Lưu ý rằng **chúng ta có thể chấm dứt vòng lặp theo k khi phát hiện trong quá trình thực hiện hai vòng lặp trong không có biến $d[v]$ nào bị đổi giá trị**. Việc này có thể xảy ra đối với $k < n-2$, và điều đó làm tăng hiệu quả của thuật toán trong việc giải các bài toán thực tế. Tuy nhiên, cải tiến đó không thực sự cải thiện được đánh giá độ phức tạp của bản thân thuật toán.

Ví dụ 3.4. Xét đồ thị trong hình dưới đây. Tìm đường đi ngắn nhất từ đỉnh $s=1$ đến các đỉnh còn lại.



A=

∞	2	∞	∞	∞
∞	∞	3	3	8
∞	∞	∞	1	-5
∞	∞	∞	∞	∞
∞	∞	∞	4	∞

Các kết quả tính toán theo thuật toán được mô tả trong bảng dưới đây:

k	d[1] Truoc[1]	d[2] Truoc[2]	d[3] Truoc[3]	d[4] Truoc[4]	d[5] Truoc[5]
	0,1	1,1	∞ ,1	∞ ,1	3,1
1	0,1	1,1	4,2	4,2	-1,3
2	0,1	1,1	4,2	3,5	-1,3
3	0,1	1,1	4,2	3,5	S

Bảng kết quả tính toán theo thuật toán Ford_Bellman

Trong các mục tiếp theo chúng ta sẽ xét một số trường hợp riêng của bài toán tìm đường đi ngắn nhất mà để giải chúng có thể xây dựng những thuật toán hiệu quả

hơn thuật toán Ford_Bellman. Đó là khi trọng số của tất cả các cung là các số không âm hoặc là khi đồ thị không có chu trình.

3.2.2 Thuật toán Dijkstra – Trường hợp đồ thị không có cạnh/cung âm.

Trong trường hợp trọng số trên các cung là không âm thuật toán do Dijkstra đề nghị làm việc hữu hiệu hơn rất nhiều so với thuật toán trình bày trong mục trước. Thuật toán được xây dựng dựa trên cơ sở gán cho các đỉnh các nhãn tạm thời. Nhãn của mỗi đỉnh cho biết cận của độ dài đường đi ngắn nhất từ s đến nó. Các nhãn này sẽ được biến đổi theo một thủ tục lặp, mà ở mỗi bước lặp có một nhãn tạm thời trở thành nhãn cố định. Nếu nhãn của một đỉnh nào đó trở thành một nhãn cố định thì nó sẽ cho ta không phải là cận trên mà là độ dài của đường đi ngắn nhất từ đỉnh s đến nó. Thuật toán được mô tả cụ thể như sau:

Thuật toán Dijkstra:

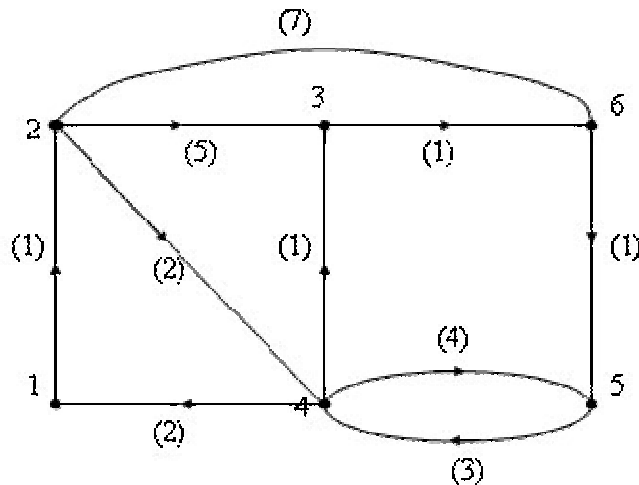
```

Begin
  (* Khởi tạo *)
  for  $v \in V$  do
    Begin
       $d[v] := a[s, v];$ 
       $Truoc[v] := s;$ 
    End;
   $d[s] := 0;$     $T := V \setminus \{s\};$    (*  $T$  là tập các đỉnh có nhãn tạm thời *)

  (* Bước lặp *)
  while  $T \neq \emptyset$  do
    Begin
      Tìm đỉnh  $u \in T$  thoả mãn  $d[u] = \min_{z \in T} d[z];$ 
       $T := T \setminus \{u\};$    (* Cố định nhãn của đỉnh  $u$  *)
      For  $v \in T$  do
        If  $d[v] > d[u] + a[u, v]$  then
          Begin
             $d[v] := d[u] + a[u, v];$ 
             $Truoc[v] := u;$ 
          End;
        End;
      End;
    End;
  End;

```

Ví dụ 3.5. Tìm đường đi ngắn nhất từ 1 đến các đỉnh còn lại của đồ thị ở hình dưới đây.



Kết quả tính toán theo thuật toán được trình bày theo bảng dưới đây. Qui ước viết hai thành phần của nhãn theo thứ tự: $d[v]$. Đỉnh được đánh dấu * là đỉnh được chọn để cố định nhãn ở bước lặp đang xét, nhãn của nó không biến đổi ở các bước tiếp theo, vì thế ta đánh dấu -.

Bước lặp	Đỉnh 1	Đỉnh 2	Đỉnh 3	Đỉnh 4	Đỉnh 5	Đỉnh 6
Khởi tạo	0,1	1,1*	∞ ,1	∞ ,1	∞ ,1	∞ ,1
1	-	-	6,2	3,2*	∞ ,1	8,2
2	-	-	4,4*	-	7,4	8,2
3	-	-		-	7,4	5,3*
4	-	-		-	6,6*	-
5						

Lưu ý:

- Nếu chỉ cần tìm đường đi ngắn nhất từ s đến một đỉnh t nào đó thì có thể kết thúc thuật toán khi đỉnh t trở thành có nhãn cố định.
- Tương tự như trong mục 2, dễ dàng mô tả thuật toán trong trường hợp đồ thị cho bởi danh sách kề. Để có thể giảm bớt khối lượng tính toán trong việc xác định đỉnh u ở mỗi bước lặp, có thể sử dụng thuật toán Heapsort. Khi đó có thể thu được thuật toán với độ phức tạp tính toán là $O(m \log n)$.

Chương 4. Cây

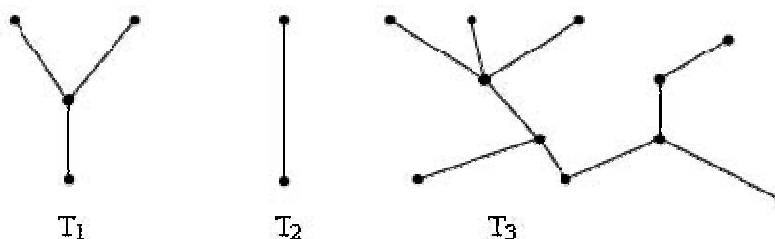
Đồ thị vô hướng liên thông không có chu trình gọi là cây. Khái niệm cây lần đầu tiên được Cayley đưa ra vào năm 1857, khi ông sử dụng chúng để đếm một dạng cấu trúc phân tử của các hợp chất hoá học trong hoá học hữu cơ. Cây còn được sử dụng rộng rãi trong rất nhiều lĩnh vực khác nhau, đặc biệt trong tin học, cây được sử dụng để xây dựng các thuật toán tổ chức các thư mục, các thuật toán cắt giữ, truyền dữ liệu và tìm kiếm...

4.1 Định nghĩa và tính chất của cây

Định nghĩa 4.1. Ta gọi **cây** là đồ thị vô hướng liên thông không có chu trình. Đồ thị không có chu trình được gọi là **rừng**.

Như vậy, rừng là đồ thị mà mỗi thành phần liên thông của nó là một cây.

Ví dụ 4.1. Trong hình 1 là một rừng gồm 3 cây T_1, T_2, T_3 .



Hình 4.1. Rừng gồm 3 cây T_1, T_2, T_3 .

Có thể nói cây là đồ thị vô hướng đơn giản nhất. Định lý sau đây cho ta một số tính chất của cây.

Định lý 4.1. Giả sử $G=(V,E)$ là đồ thị vô hướng n đỉnh. Khi đó các mệnh đề sau đây là tương đương:

- (1) T là cây;
- (2) T không chứa chu trình và có $n-1$ cạnh;
- (3) T liên thông và có $n-1$ cạnh;
- (4) T liên thông và mỗi cạnh của nó đều là cầu;
- (5) Hai đỉnh bất kỳ của T được nối với nhau bởi đúng một đường đi đơn;
- (6) T không chứa chu trình nhưng nếu thêm vào một cạnh ta thu được đúng một chu trình.

Chứng minh. Ta sẽ chứng minh định lý theo sơ đồ sau:

$$(1) \Rightarrow (2) \Rightarrow (3) \Rightarrow (4) \Rightarrow (5) \Rightarrow (6) \Rightarrow (1)$$

- $(1) \Rightarrow (2)$. Theo định nghĩa T không chứa chu trình. Ta sẽ chứng minh bằng qui nạp theo số đỉnh n cho khẳng định: Số cạnh của cây với n đỉnh là n-1. Rõ ràng khẳng định đúng với n=1. Giả sử n>1. Trước hết nhận rằng trong mọi cây T có n đỉnh đều tìm được ít nhất một đỉnh là đỉnh treo (tức là đỉnh có bậc là 1). Thực vậy, gọi $v_1, v_2, \dots, v_k$ là đường đi dài nhất (theo số cạnh) trong T. Khi đó rõ ràng v_1 và v_k là các đỉnh treo, vì từ v_1 (v_k) không có cạnh nối với bất cứ đỉnh nào trong số các đỉnh $v_2, v_3, \dots, v_k$ (do đồ thị không chứa chu trình), cũng như với bất cứ đỉnh nào khác của đồ thị (do đường đi đang xét dài nhất). Loại bỏ v_1 và cạnh (v_1, v_2) khỏi T ta thu được cây T_1 với n-1 đỉnh, mà theo giả thiết qui nạp có n-2 cạnh. Vậy cây T có n-2+1 = n-1 cạnh.
- $(2) \Rightarrow (3)$ Ta chứng minh bằng phản chứng. Giả sử T không liên thông. Khi đó T phân rã thành $k \geq 2$ phần liên thông $T_1, T_2, \dots, T_k$. Do T không chứa chu trình nên mỗi T_i ($i=1, 2, \dots, k$) cũng không chứa chu trình, vì thế mỗi T_i là cây. Do đó nếu gọi $n(T_i)$ và $e(T_i)$ theo thứ tự là số đỉnh và cạnh của T_i , ta có:

$$e(T_i) = n(T_i) - 1, i = 1, 2, \dots, k,$$
 Suy ra

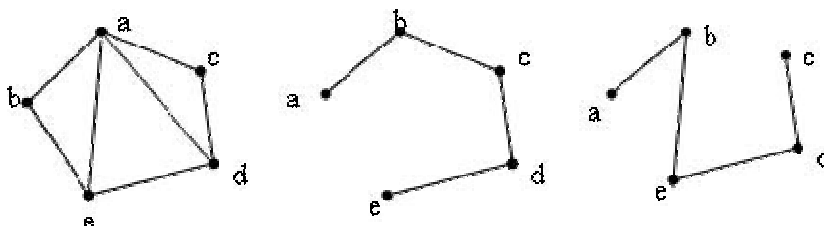
$$\begin{aligned} n-1 &= e(T) = e(T_1) + \dots + e(T_k) \\ &= n(T_1) + \dots + n(T_k) - k \\ &= n(T) - k < n-1 \end{aligned}$$
 Mâu thuẫn thu được chứng tỏ là T liên thông.
- $(3) \Rightarrow (4)$ Việc loại bỏ một cạnh bất kỳ khỏi T dẫn đến đồ thị với n đỉnh và n-2 cạnh rõ ràng là đồ thị không liên thông. Vậy mọi cạnh trong T đều là cầu.
- $(4) \Rightarrow (5)$ Do T là liên thông nên hai đỉnh bất kỳ của nó được nối với nhau bởi một đường đi đơn. Nếu có cặp đỉnh nào của T có hai đường đi đơn khác nhau nối chúng, thì từ đó suy ra đồ thị chứa chu trình, và vì thế các cạnh trên chu trình này không phải là cầu.
- $(5) \Rightarrow (6)$ T không chứa chu trình, bởi vì thế nếu có chu trình thì hoá ra tìm được cặp đỉnh của T được nối với nhau bởi hai đường đi đơn. Bây giờ, nếu thêm vào T một cạnh e nối hai đỉnh u và v nào đó của T. Khi đó cạnh này cùng với đường đi đơn nối u với v sẽ tạo thành chu trình trong T. Chu trình thu được này là duy nhất, vì nếu thu được nhiều hơn một chu trình thì suy ra trong T trước đó phải có sẵn chu trình.
- $(6) \Rightarrow (1)$ Giả sử T không liên thông. Khi đó gồm ít ra là 2 thành phần liên thông. Vì vậy, nếu thêm vào T một cạnh nối hai đỉnh thuộc hai thành phần liên thông khác nhau ta không thu được thêm một chu trình nào cả. Điều đó mâu thuẫn với giả thiết (6).

Định lý được chứng minh. ■

4.2 Cây khung và bài toán cây khung nhỏ nhất

Định nghĩa 4.2. Cho $G=\langle V,E \rangle$ là đồ thị vô hướng liên thông. Cây $T=\langle V,F \rangle$ với $F\subseteq E$ được gọi là **cây khung** (spanning tree) của đồ thị G .

Ví dụ 4.2. Đồ thị G và cây khung của nó được cho trong hình 4.2.



Hình 4.2. Đồ thị và các cây khung của nó

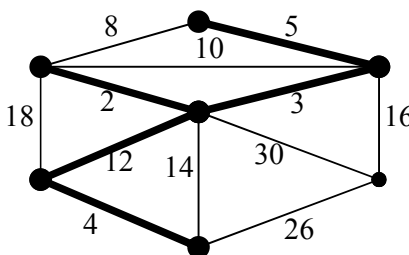
Một đồ thị có thể có rất nhiều cây khung khác nhau. Chúng ta có thể dùng các thuật toán tìm kiếm theo chiều sâu hay tìm kiếm theo chiều rộng để tìm cây khung của đồ thị. Trong số các cây khung của đồ thị, ta quan tâm tới một dạng cây khung đặc biệt: cây khung có tổng trọng số các cạnh là nhỏ nhất.

Bài toán cây khung nhỏ nhất:

Đây là một trong những bài toán tối ưu trên đồ thị. Bài toán này được ứng dụng trong nhiều lĩnh vực khác nhau của đời sống. Trước hết, ta phát biểu nội dung của bài toán:

Cho $G = \langle V,E \rangle$ là đồ thị vô hướng liên thông và có trọng số, với tập đỉnh $V = \{1,2,3,4,\dots,n\}$ và tập E gồm m cạnh. Giả sử $H=\langle V,T \rangle$ là cây khung của đồ thị, ta gọi $c(H)$, là **độ dài của cây khung**, bằng tổng trọng số trên các cạnh của nó. Bài toán đặt ra là trong số các cây khung của G , hãy tìm cây khung có độ dài nhỏ nhất. Cây khung tìm được gọi là **cây khung nhỏ nhất** của đồ thị.

Ví dụ 4.3. Hình vẽ sau đây cho ta một ví dụ về một đồ thị vô hướng liên thông có trọng số và cây khung nhỏ nhất của nó (các cạnh của cây khung được in đậm) với tổng độ dài là: $5 + 2 + 3 + 12 + 4 = 26$.



Để giải bài toán cây khung nhỏ nhất, ta có thể liệt kê tất cả các cây khung của đồ thị và chọn trong số chúng cây khung nhỏ nhất. Tuy nhiên cách này có thời gian tính toán rất lâu. Chúng ta sẽ khảo sát hai thuật toán hiệu quả hơn rất nhiều: Thuật toán Kruskal và thuật toán Prim.

4.2.1 Thuật toán Kruskal:

Thuật toán sẽ xây dựng tập cạnh của cây khung nhỏ nhất $H=<V,T>$ theo từng bước. Để ý rằng, những cạnh có trọng số nhỏ thường nằm trong cây khung nhỏ nhất. Theo ý tưởng này, chúng ta sẽ khảo sát lần lượt các cạnh có trọng số từ nhỏ tới lớn và thử việc đưa chúng vào cây khung, nếu không được (tạo với những cạnh đã chọn thành chu trình) lại chọn cạnh có trọng số lớn hơn, ... quá trình cứ tiếp tục cho đến khi tìm được đủ cạnh cho cây khung nhỏ nhất.

Cụ thể, thuật toán được mô tả như sau:

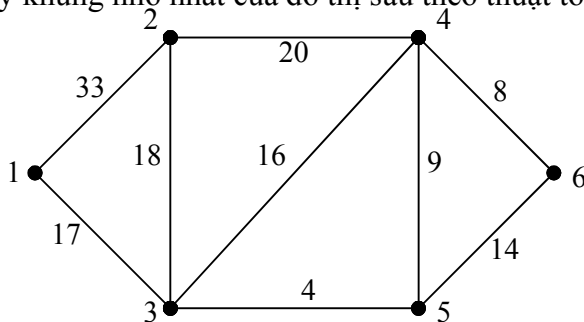
Thuật toán Kruskal:

```

Begin
   $T := \emptyset$ ;
  While  $|T| < (n-1)$  and  $(E \neq \emptyset)$  do
    Begin
      Chọn e là cạnh độ dài nhỏ nhất trong E
       $E := E \setminus \{e\}$ 
      If  $(T \cup \{e\}$  không chứa chu trình) then
         $T := T \cup \{e\}$     (* Kết nạp cạnh e vào cây khung nhỏ nhất *)
    End;
  If  $(|T| < (n-1))$  then
    Đồ thị không liên thông.
  End;

```

Ví dụ 4.4. Tìm cây khung nhỏ nhất của đồ thị sau theo thuật toán Kruskal:



Bước khởi tạo. Đặt $T := \emptyset$; Sắp xếp các cạnh của đồ thị theo thứ tự tăng dần về trọng số, ta có:

(3,5), (4,6), (4,5), (5,6), (3,4), (1,3), (2,3), (2,4), (1,2)

Đầu tiên, ta chọn cạnh (3,5) đưa vào cây. Không có vấn đề gì.

Kế tiếp, chọn cạnh (4,6) đưa vào cây. Không có vấn đề gì.

Kế tiếp, chọn cạnh (4,5) đưa vào cây. Không có vấn đề gì.

Kế tiếp, chọn cạnh (5,6) đưa vào cây. Không được do tạo thành chu trình 4 5 6

4.

Kế tiếp, chọn cạnh (3,4) đưa vào cây. Không được do tạo thành chu trình 3 5 4

3.

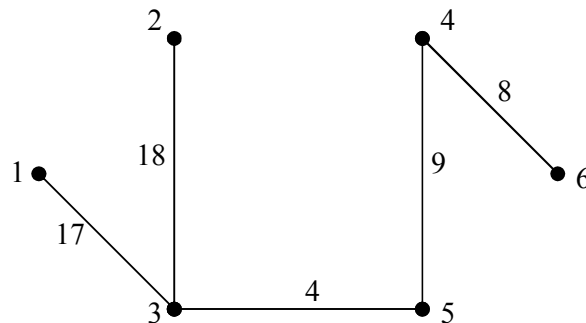
Kế tiếp, chọn cạnh (1,3) đưa vào cây. Không có vấn đề gì.

Kế tiếp, chọn cạnh (2,3) đưa vào cây. Không có vấn đề gì.

Kết thúc vì đã lấy đủ cạnh.

Tập cạnh của cây tối đại cần tìm là: $T = \{(3,5), (4,5), (4,5), (1,3), (2,3)\}$

Khi đó cây cần tìm sẽ có dạng:



4.2.2 Thuật toán Prim

Thuật toán Kruskal là việc kém hiệu quả đối với các đồ thị có nhiều cạnh. Trong khi đó, đối với trường hợp này, thuật toán Prim tỏ ra hiệu quả hơn. Thuật toán Prim còn được gọi là phương pháp lân cận gần nhất. Trong phương pháp này, bắt đầu từ một đỉnh tùy ý s của đồ thị, đầu tiên, ta nối s với đỉnh lân cận gần nó nhất, chẳng hạn là đỉnh t . Nghĩa là trong số các cạnh nối với s , cạnh (s,t) có trọng số nhỏ nhất. Kế tiếp, trong số các cạnh nối với s hay t , ta lại chọn cạnh có trọng số nhỏ nhất, ... cho đến khi ta thu được cây gồm n đỉnh và $n-1$ cạnh. Đây chính là cây khung nhỏ nhất cần tìm.

Để biểu diễn lời giải, ta sẽ sử dụng 2 mảng:

- Mảng $d[v]$ dùng để lưu độ dài cạnh ngắn nhất nối với v trong số các cạnh chưa xét.

- Mảng near: near[v] dùng để lưu đỉnh còn lại (ngoài v) của cạnh gần nhất nối ở trên.

Thuật toán Prim:**Begin**

(* Khởi tạo *)

Chọn s là một đỉnh nào đó của đồ thị

 $V_H := \{s\};$ (* Tập những đỉnh đã đưa vào cây *) $T := \emptyset;$ (* Tập cạnh của cây *) $d[s] = 0;$ near[s] = s;**For** $v \in V \setminus V_H$ **do****Begin** $d[v] := a[s, v];$

near[v] := s;

End;

(* Bước lặp *)

Stop := False;

While (not Stop) **do****Begin**Tìm $u \in V \setminus V_H$ thỏa mãn $d[u] = \min\{d[v] : v \in V \setminus V_H\};$ $V_H := V_H \cup \{u\};$ $T := T \cup \{(u, \text{near}[u])\};$ **If** $|V_H| = n$ **then****Begin** $H := (V_H, T)$ là cây khung của đồ thị.

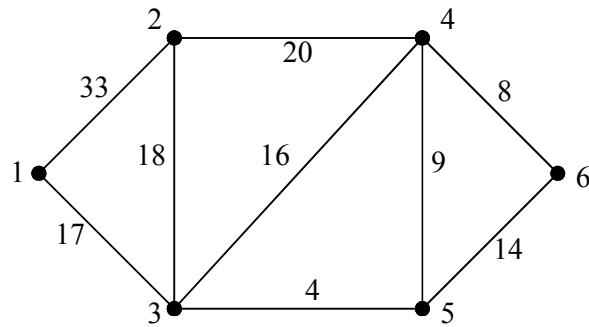
Stop := True;

End;**Else****For** $v \in V \setminus V_H$ **do****If** $d[v] > a[u, v]$ **then****Begin** $d[v] := a[u, v];$

near[v] := u;

End;**End;****End;**

Ví dụ 4.5. Tìm cây khung nhỏ nhất của đồ thị sau theo thuật toán Prim:



Bảng dưới đây ghi nhận các đỉnh trong bước lặp của thuật toán, đỉnh đánh dấu * là đỉnh được chọn để bổ sung vào cây khung (khi đó nhãn của nó không còn bị biến đổi trong các bước lặp tiếp theo, vì vậy ta đánh dấu – để ghi nhận điều đó):

Bước lặp	Đỉnh 1	Đỉnh 2	Đỉnh 3	Đỉnh 4	Đỉnh 5	Đỉnh 6	V_H	T
Khởi tạo	[0,1]	[33,1]	[17,1]*	$[\infty,1]$	$[\infty,1]$	$[\infty,1]$	1	$\emptyset$
1	-	[18,3]	-	[16,3]	[4,3]*	$[\infty,1]$	1,3	(3,1)
2	-	[18,3]	-	[9,5]*	-	[14,5]	1,3,5	(3,1),(5,3)
3	-	[18,3]	-	-	-	[8,4]*	1,3,5,4	(3,1),(5,3),(4,5)
4	-	[18,3]*	-	-	-	-	1,2,3,4,6	(3,1),(5,3),(4,5) (6,4)
5	-	-	-	-	-	-	1,2,3,4,6,2	(3,1),(5,3),(4,5) (6,4),(2,3)

TÀI LIỆU THAM KHẢO

- [1]. Nguyễn Đức Nghĩa – Nguyễn Tô Thành, Toán rời rạc, NXB Đại học Quốc Gia Hà Nội, 2003.**
- [2]. Kenneth H.Rosen – Toán rời rạc ứng dụng trong Tin học (Bản dịch), NXB Khoa học và Kỹ thuật, 2000.**
- [3]. Nguyễn Cam – Chu Đức Khánh, Lý thuyết đồ thị - NXB Trẻ 1998.**
- [4]. Hoàng Chúng , Đại cương về Toán học hữu hạn, NXB Giáo Dục, 1999.**